Instituto Tecnológico de Pachuca

Migración y publicación de una API para vehículo IoT en AWS

Asignatura: Implementación de Soluciones I

Carrera: Ingeniería en sistemas computacionales

Alumno: Lopez García Juan Elpidio

Maestro: Pinedo Fernández Víctor Manuel

Semestre: 9°

Repositorio: https://github.com/juanel2804/vehiculo-iot-api

Fecha de entrega:
21/10/2026

Introducción

En las prácticas 2.3 y 2.4 se diseñaron las seis tablas y los 21 Stored Procedures del esquema `vehiculo_iot` y se migraron a Amazon RDS for MySQL. En la práctica 2.5 se desarrolló localmente una API en Python con FastAPI y programación orientada a objetos que usa esos 21 SP, expone servicios GET y POST, documentación Swagger UI y un canal WebSocket por vehículo. En este examen esa misma API se migró a una instancia Amazon EC2 y se publicó por HTTPS y WebSocket seguro (WSS), de modo que cualquier cliente externo puede enviar comandos al vehículo y observar sus movimientos en tiempo real sin usar `localhost`.

Objetivo: migrar e implementar de manera individual la API Python/FastAPI de la práctica 2.5 hacia un entorno público en Amazon EC2, utilizando programación orientada a objetos y Amazon RDS for MySQL; integrar los 21 procedimientos almacenados, los servicios GET y POST, Swagger UI y WebSocket seguro (WSS), y demostrar que el WebSocket transmite el último estado, notifica los movimientos individuales enviados por POST y presenta en tiempo real los N movimientos de una DEMO.

Datos de la práctica

API pública (HTTPS)	https://100-63-187-92.sslip.io
Swagger UI	https://100-63-187-92.sslip.io/docs
WebSocket seguro (WSS)	`wss://100-63-187-92.sslip.io/ws/vehiculos/{id_vehiculo}` (vehículo 1: `wss://100-63-187-92.sslip.io/ws/vehiculos/1`)
Página de control	https://100-63-187-92.sslip.io/control/
Repositorio GitHub	https://github.com/juanel2804/vehiculo-iot-api (commit `f634f3c`, versión 2.6.0)
Servidor	Amazon EC2 `WebServer`, Windows Server 2025 Datacenter, t3.micro, IP elástica 100.63.187.92
HTTPS / WSS	Caddy como proxy inverso con certificado Let's Encrypt automático; dominio `100-63-187-92.sslip.io`
Ejecución persistente	Tareas de inicio de Windows `IoT-API` (uvicorn en 127.0.0.1:8000) e `IoT-Caddy`, cuenta SYSTEM
Base de datos	Amazon RDS for MySQL 8.4.9, instancia `instancia-rds-iot`, esquema `vehiculo_iot` (6 tablas, 21 SP)
Usuario de MySQL	`api_iot`, solo con privilegio EXECUTE sobre `vehiculo_iot`
Tecnologías	Python 3.14, FastAPI, Uvicorn, PyMySQL, Pydantic, websockets, Git/GitHub
Credenciales	Variables de entorno en `.env`, que existe solo en la EC2 y está excluido de Git; la `X-API-Key` aparece oculta en todas las capturas

1. Preparación de AWS y migración del proyecto

Paso 1. Arquitectura pública EC2–RDS

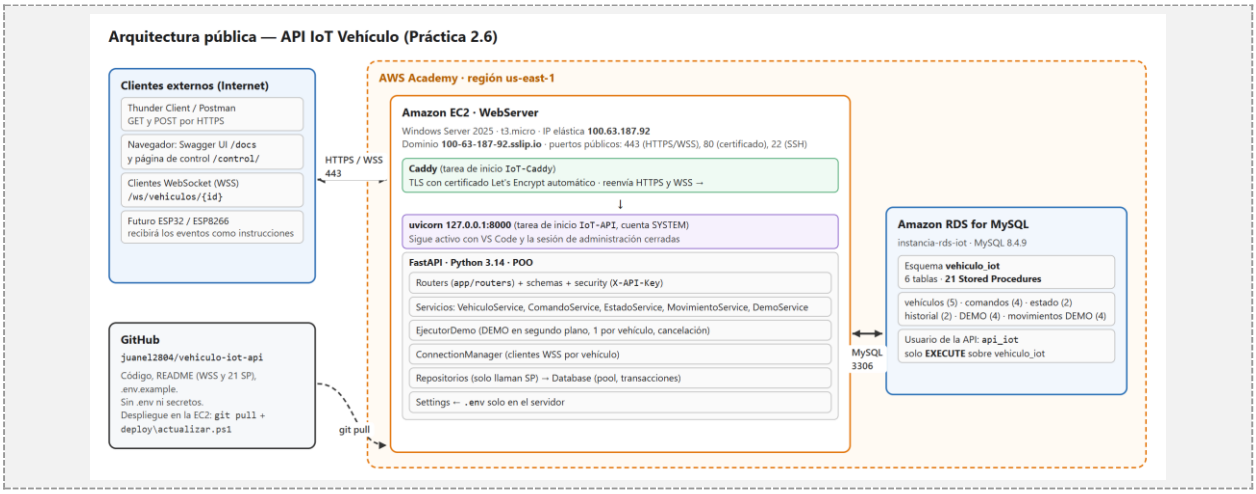


Figura 1. *Arquitectura pública de la solución: clientes externos, Amazon EC2 con Caddy y FastAPI, y Amazon RDS for MySQL.*

Los clientes externos (Thunder Client, Postman, el navegador con Swagger UI y los clientes WebSocket) llegan por Internet al puerto 443 de la EC2. Ahí Caddy termina el cifrado TLS con un certificado de Let's Encrypt y reenvía tanto las peticiones HTTPS como las conexiones WSS a uvicorn, que solo escucha en `127.0.0.1:8000`, por lo que la aplicación no queda expuesta sin cifrado. La API se conecta a Amazon RDS por MySQL 3306 con el usuario `api_iot`. El código se despliega en la EC2 desde GitHub con `git pull` y el script `deploy\actualizar.ps1`.

Se conservó la misma organización en capas de la práctica 2.5: routers delgados, servicios con las reglas de negocio, repositorios que solo llaman Stored Procedures, la clase `Database` con el pool de conexiones y las transacciones, `EjecutorDemo` para las DEMO y `ConnectionManager` para los clientes WebSocket.

Paso 2. Ejecución persistente en Amazon EC2

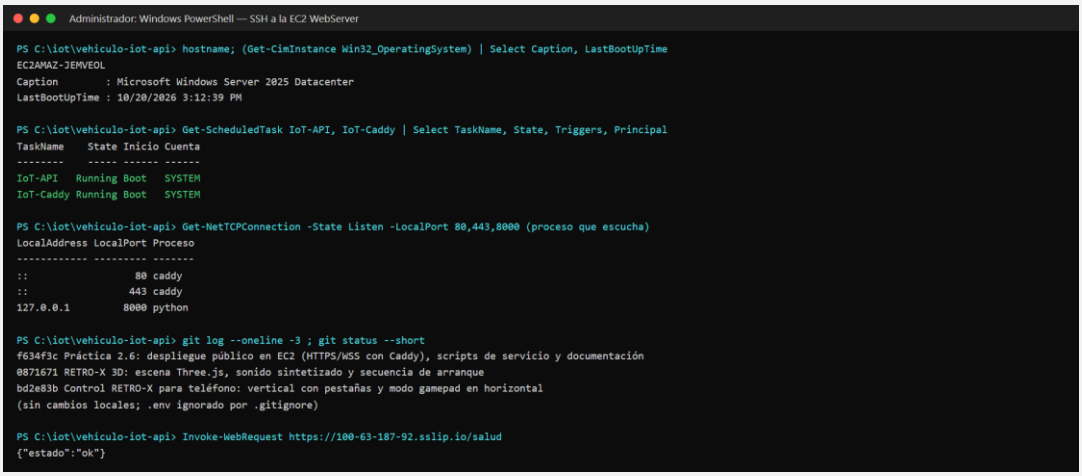


Figura 2. *Estado del servidor consultado por SSH: tareas de inicio IoT-API e IoT-Caddy en ejecución, puertos en escucha y commit desplegado.*

La API no depende de Visual Studio Code ni de una sesión abierta. Las tareas programadas `IoT-API` e `IoT-Caddy` se ejecutan al arrancar Windows (disparador *Boot*) con la cuenta SYSTEM. La captura se tomó por SSH el 20/10/2026 después de encender la instancia: el sistema arrancó a las 15:12 UTC y ambas tareas ya estaban en ejecución sin que nadie iniciara sesión. Caddy escucha en los puertos 80 y 443 de todas las interfaces y Python (uvicorn) solo en `127.0.0.1:8000`. El repositorio de la EC2 está en el commit `f634f3c` de GitHub, sin cambios locales, y `/salud` responde `{"estado":"ok"}` por HTTPS.

Paso 3. Comunicación con Amazon RDS y los 21 Stored Procedures

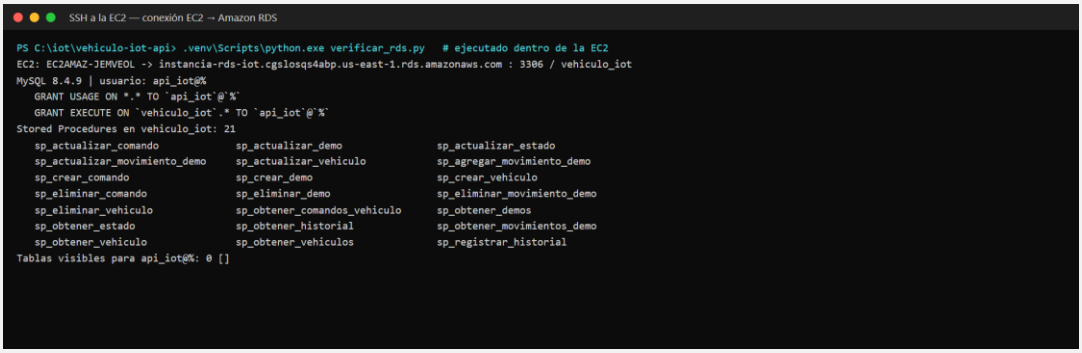


Figura 3. *Desde la EC2: conexión a Amazon RDS con el usuario api_iot, sus permisos y los 21 Stored Procedures de vehiculo_iot.*

Para comprobar la conectividad se ejecutó, dentro de la EC2, un script que usa la misma configuración de la API. La conexión llegó a `instancia-rds-iot` (MySQL 8.4.9) con el usuario `api_iot@%`, que solo tiene `USAGE` global y `EXECUTE` sobre `vehiculo_iot`. Con ese permiso el usuario ve los **21 Stored Procedures**: vehículos (5), comandos (4), estado (2), historial (2), DEMO (4) y movimientos DEMO (4). En cambio, no puede ver ni consultar las tablas directamente (aparecen 0

tablas visibles): la API solo accede a las seis tablas a través de los procedimientos, que conservan sus nombres y parámetros originales.

2. Publicación de servicios HTTPS y documentación

Paso 4. Swagger UI público por HTTPS

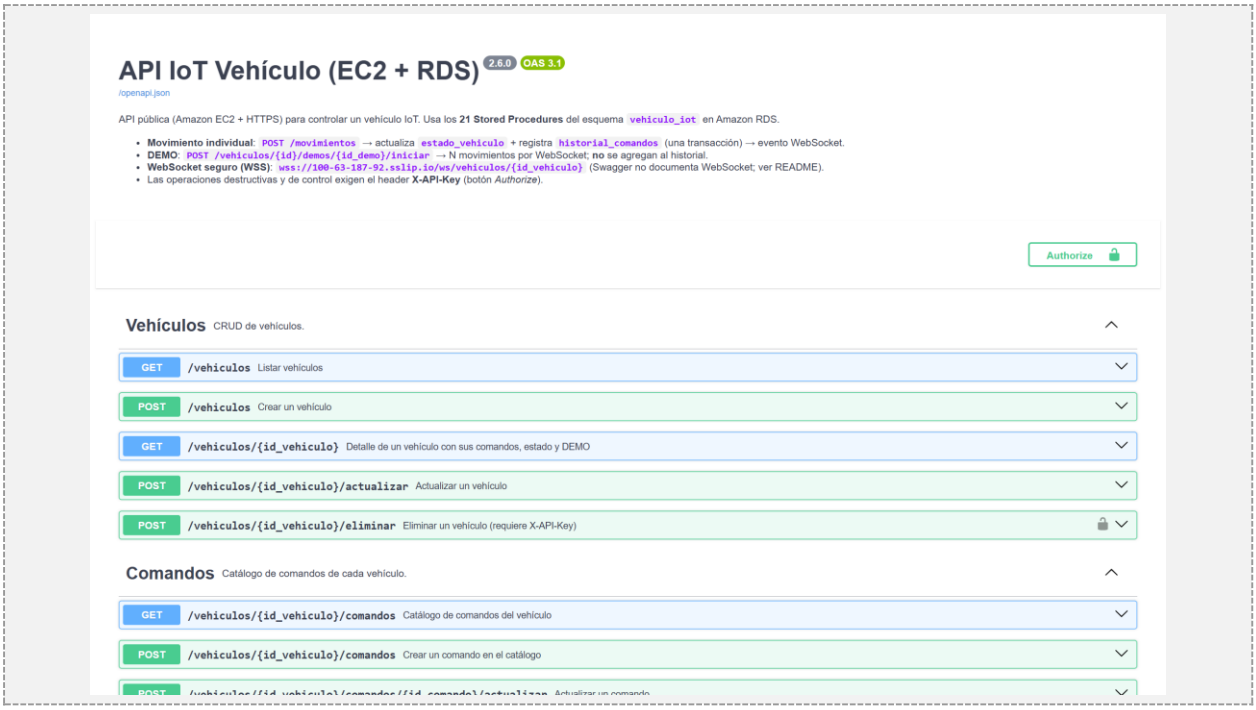


Figura 4. *Swagger UI en https://100-63-187-92.sslip.io/docs (versión 2.6.0) con la descripción de los flujos y la URL del WSS.*

La documentación se publica en https://100-63-187-92.sslip.io/docs. La descripción general explica el movimiento individual (`POST /movimientos`, que actualiza `estado_vehiculo` y registra `historial_comandos` en una transacción), la DEMO (`POST /vehiculos/{id}/demos/{id_demo}/iniciar`, que transmite N movimientos por WebSocket sin agregarlos al historial), la dirección del WebSocket seguro y el uso del header `X-API-Key` mediante el botón *Authorize*.

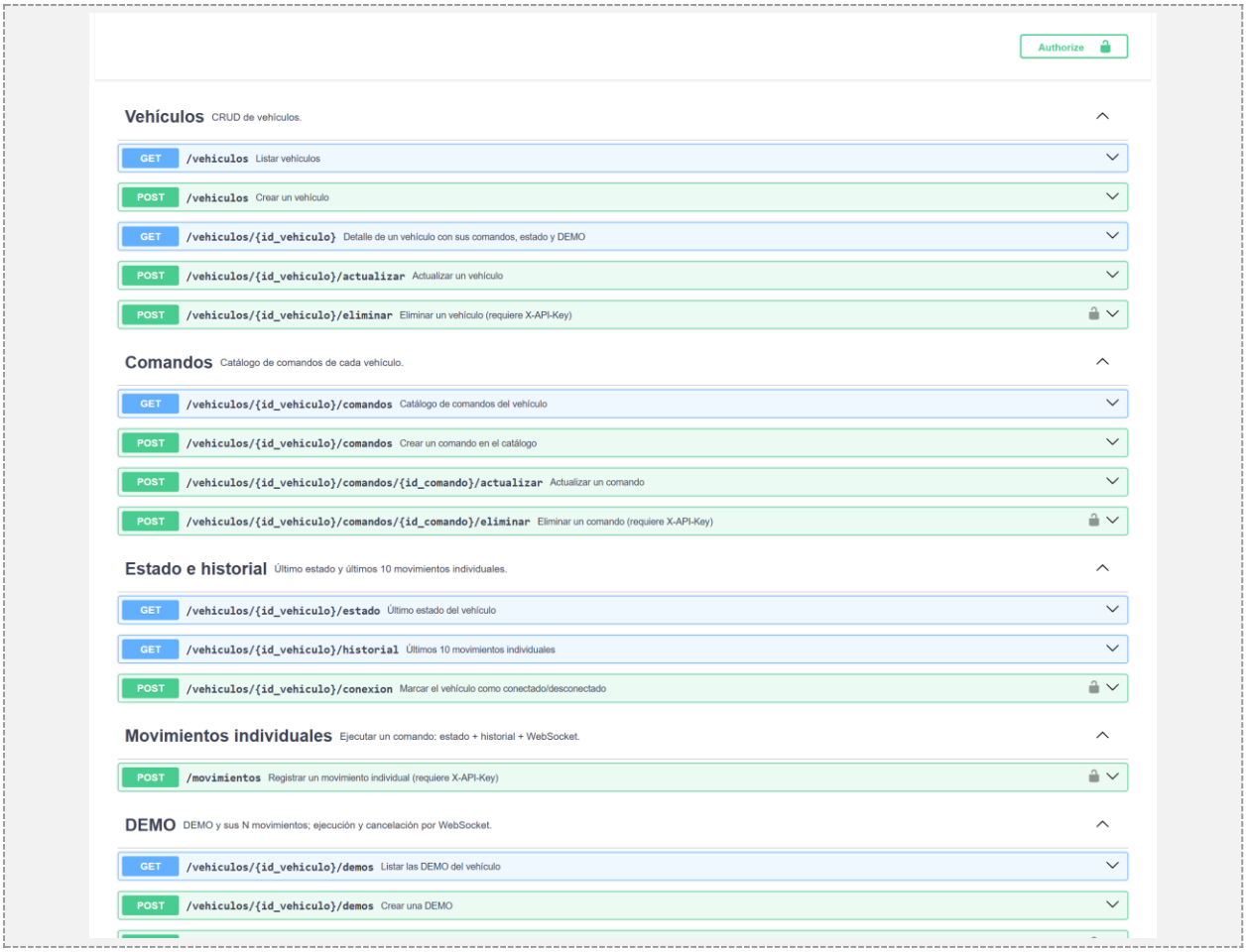


Figura 5. *Endpoints agrupados por etiqueta; las operaciones de control y eliminación muestran el candado de X-API-Key.*

Los endpoints están agrupados en Vehículos, Comandos, Estado e historial, Movimientos individuales, DEMO y Sistema. Cada operación documenta sus parámetros, el modelo de respuesta, los códigos de error y el Stored Procedure que utiliza. Las consultas usan GET y nunca modifican datos; las altas, actualizaciones, eliminaciones y el control usan POST.

Paso 5. Repositorio GitHub y documentación del WebSocket

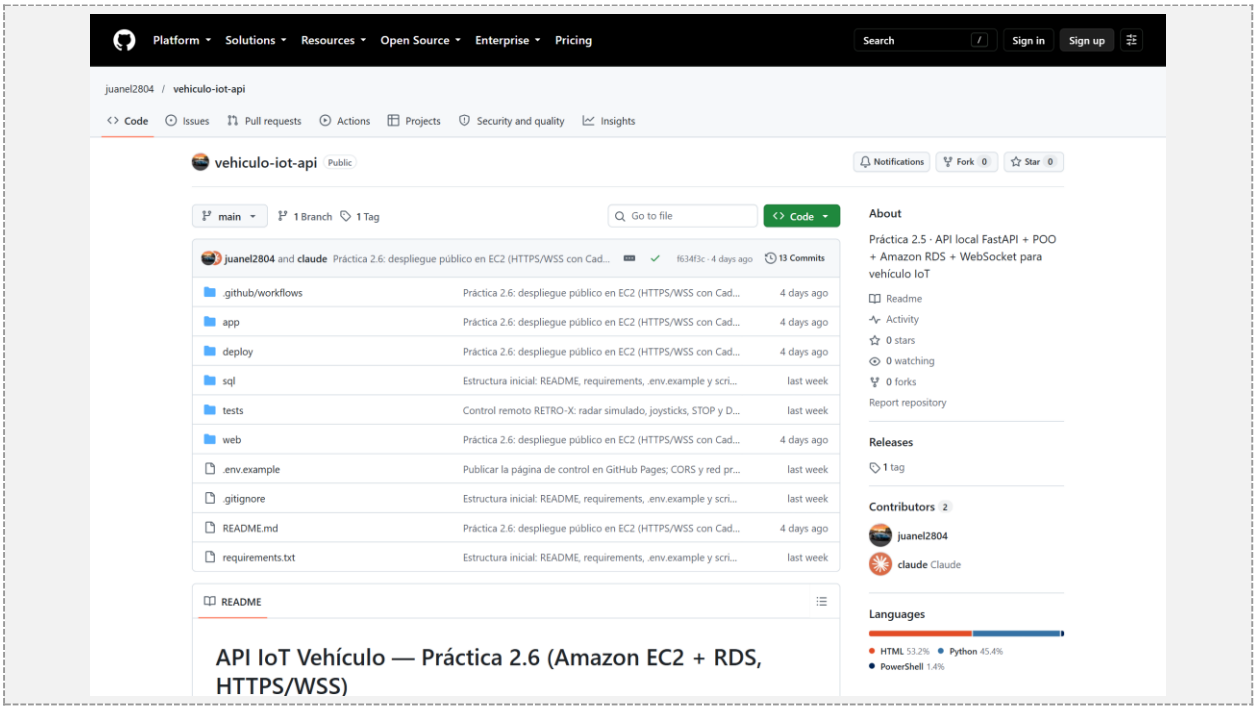
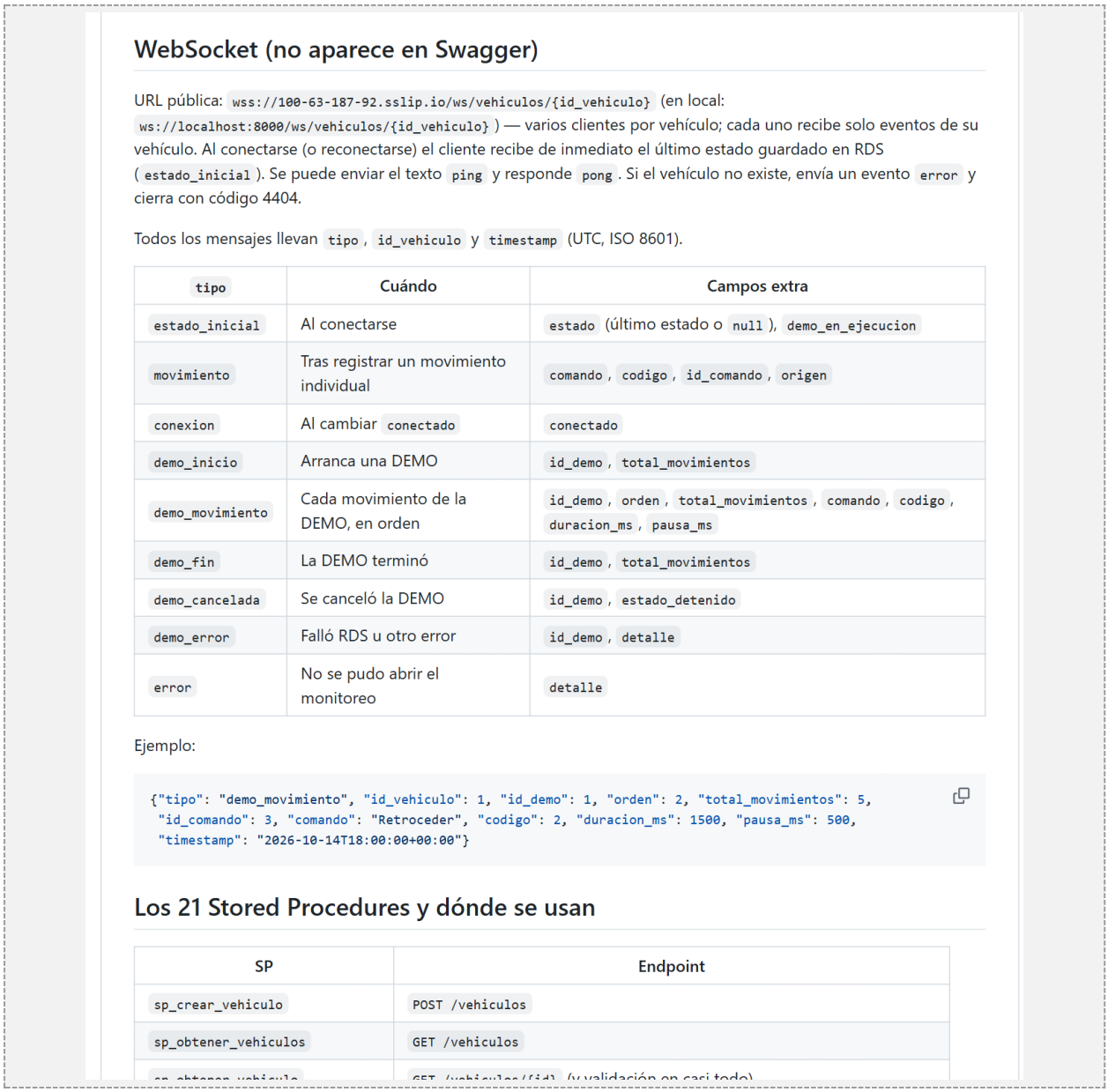


Figura 6. *Repositorio juanel2804/vehiculo-iot-api con el código, la carpeta deploy y el README de la práctica 2.6.*

El código está en https://github.com/juanel2804/vehiculo-iot-api. El repositorio incluye los módulos de la API (`app/`), `requirements.txt`, los scripts SQL del usuario de la API, la prueba integral (`tests/`), la carpeta `deploy/` (Caddyfile de ejemplo, instalación de las tareas de inicio y actualización desde GitHub) y `.env.example` sin secretos. `.gitignore` excluye `.env`, `.venv` y las cachés. La etiqueta `v2.5-entregada` marca lo entregado en la práctica anterior.



WebSocket (no aparece en Swagger)

URL pública: `wss://100-63-187-92.sslip.io/ws/vehiculos/{id_vehiculo}` (en local: `ws://localhost:8000/ws/vehiculos/{id_vehiculo}`) — varios clientes por vehículo; cada uno recibe solo eventos de su vehículo. Al conectarse (o reconectarse) el cliente recibe de inmediato el último estado guardado en RDS (`estado_inicial`). Se puede enviar el texto `ping` y responde `pong`. Si el vehículo no existe, envía un evento `error` y cierra con código 4404.

Todos los mensajes llevan `tipo`, `id_vehiculo` y `timestamp` (UTC, ISO 8601).

`tipo`	Cuándo	Campos extra
`estado_inicial`	Al conectarse	`estado` (último estado o `null`), `demo_en_ejecucion`
`movimiento`	Tras registrar un movimiento individual	`comando`, `codigo`, `id_comando`, `origen`
`conexion`	Al cambiar `conectado`	`conectado`
`demo_inicio`	Arranca una DEMO	`id_demo`, `total_movimientos`
`demo_movimiento`	Cada movimiento de la DEMO, en orden	`id_demo`, `orden`, `total_movimientos`, `comando`, `codigo`, `duracion_ms`, `pausa_ms`
`demo_fin`	La DEMO terminó	`id_demo`, `total_movimientos`
`demo_cancelada`	Se canceló la DEMO	`id_demo`, `estado_detenido`
`demo_error`	Falló RDS u otro error	`id_demo`, `detalle`
`error`	No se pudo abrir el monitoreo	`detalle`

Ejemplo:

```
{"tipo": "demo_movimiento", "id_vehiculo": 1, "id_demo": 1, "orden": 2, "total_movimientos": 5,
 "id_comando": 3, "comando": "Retroceder", "codigo": 2, "duracion_ms": 1500, "pausa_ms": 500,
 "timestamp": "2026-10-14T18:00:00+00:00"}
```

Los 21 Stored Procedures y dónde se usan

SP	Endpoint
`sp_crear_vehiculo`	`POST /vehiculos`
`sp_obtener_vehiculos`	`GET /vehiculos`
`sp_obtener_vehiculo`	`GET /vehiculos/{id}` (y validación en casi todo)

Figura 7. *Sección WebSocket del README: URL pública WSS, eventos y campos de cada mensaje JSON.*

Como Swagger no documenta WebSocket, el README describe la URL pública `wss://100-63-187-92.sslip.io/ws/vehiculos/{id_vehiculo}`, el comportamiento al conectarse (`estado_inicial`), el `ping`/`pong`, el cierre con código 4404 si el vehículo no existe y la tabla de eventos (`movimiento`, `conexion`, `demo_inicio`, `demo_movimiento`, `demo_fin`, `demo_cancelada`, `demo_error`, `error`) con sus campos. También relaciona cada uno de los 21 SP con su endpoint (ver Anexo A).

3. Escenarios funcionales desde un cliente externo

Las tres pruebas se ejecutaron el 20/10/2026 contra la URL pública desde una computadora externa a AWS, con un cliente HTTPS/WSS que abre dos conexiones WebSocket al vehículo 1 y envía las peticiones con el header `X-API-Key` oculto. Las horas de la columna izquierda de cada evento son

las del cliente (hora del centro de México, UTC−6) y los campos `timestamp` y `fecha_hora` de la API están en UTC; por ejemplo, 09:38:01 locales corresponden a 15:38:01 UTC. Las mismas peticiones están guardadas en la colección de Postman `Practica-2.6-API-IoT.postman_collection.json` (Figura 15).

Prueba 1. POST público y notificación WSS individual



Figura 8. *Dos clientes WSS conectados al vehículo 1: ambos reciben estado_inicial con el último estado guardado en RDS.*

Se abrieron dos conexiones a `wss://100-63-187-92.sslip.io/ws/vehiculos/1` (clientes A y B). Al establecerse, cada una recibió de inmediato el evento `estado_inicial` con el último estado leído de RDS mediante `sp_obtener_estado`: comando DETENER (código 9), `conectado` = 1 y `demo_en_ejecucion` = null.

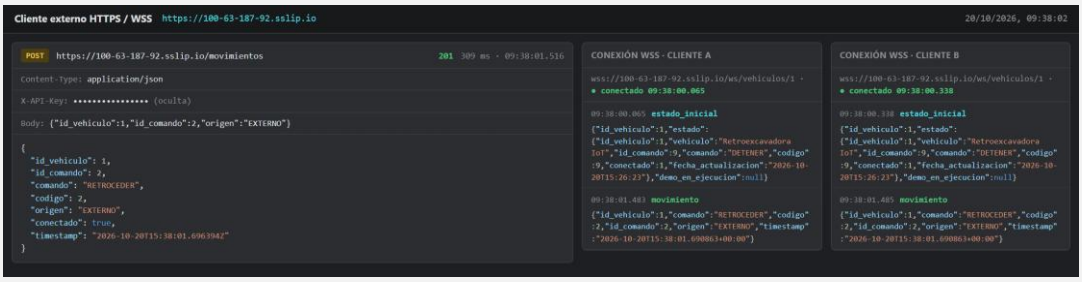


Figura 9. *POST https://100-63-187-92.sslip.io/movimientos (RETROCEDER) con respuesta 201 y el evento movimiento recibido por los dos clientes WSS.*

Con las dos conexiones abiertas se envió `POST /movimientos` con el cuerpo `{"id_vehiculo": 1, "id_comando": 2, "origen": "EXTERNO"}`. La API validó que el vehículo existe y está activo y que el comando 2 pertenece a su catálogo y está activo; después ejecutó `sp_actualizar_estado` y `sp_registrar_historial` en una sola transacción y respondió **201** con el comando RETROCEDER. Ya confirmada la transacción, ambos clientes recibieron el mismo evento `movimiento` (09:38:01.483 y 09:38:01.485) con el mismo `timestamp` del servidor.

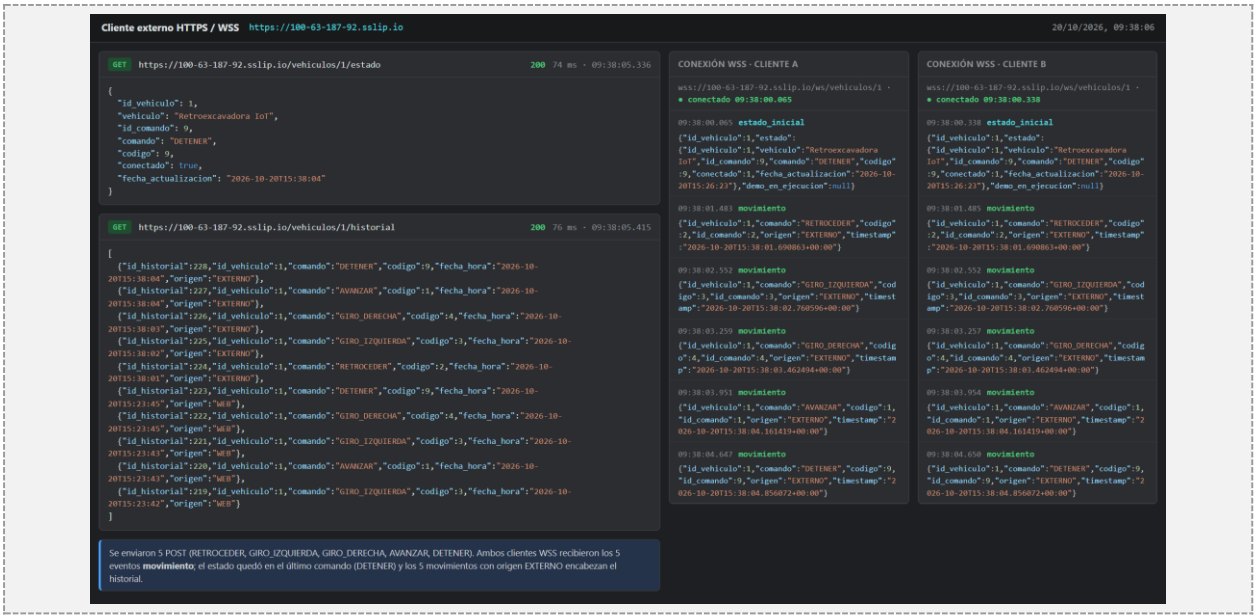


Figura 10. *Cinco POST recibidos por ambos clientes WSS; el GET de estado y el historial confirman la persistencia en RDS.*

Se repitió el POST con los comandos 3, 4, 1 y 9 (GIRO_IZQUIERDA, GIRO_DERECHA, AVANZAR y DETENER). Los dos clientes recibieron los cinco eventos `movimiento` en el mismo orden. `GET /vehiculos/1/estado` devolvió DETENER, el último comando enviado, y `GET /vehiculos/1/historial` muestra los cinco movimientos con origen EXTERNO (id_historial 224 a 228) al principio del historial, lo que demuestra que el estado y el historial quedaron guardados en RDS.

Prueba 2. GET público del historial reciente



Figura 11. *GET https://100-63-187-92.sslip.io/vehiculos/1/historial: 10 movimientos individuales, del más reciente al más antiguo.*

`GET /vehiculos/1/historial` respondió **200** con exactamente 10 movimientos individuales, aunque el vehículo tiene más registros guardados. Los `id_historial` van en orden descendente (228, 227, … 219), es decir, del más reciente al más antiguo, y ningún registro tiene origen DEMO: todos son EXTERNO (esta prueba) o WEB (enviados desde la página de control). La consulta no requiere llave, solo llama a `sp_obtener_historial` y no elimina ni modifica registros de RDS.

Prueba 3. WSS: DEMO en tiempo real sin historial

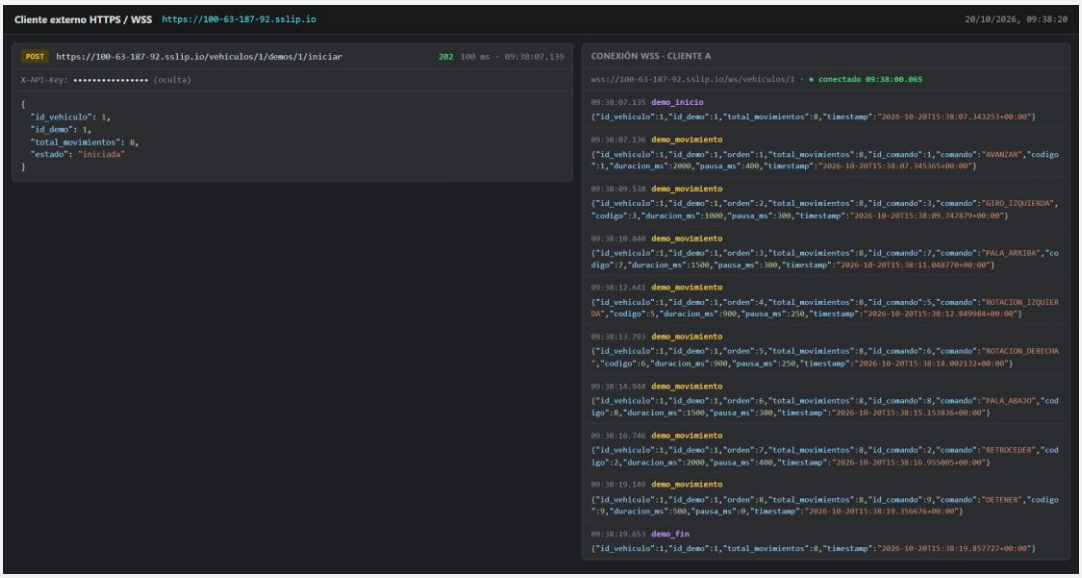


Figura 12. *POST iniciar DEMO 1 (202) y eventos WSS recibidos: demo_inicio, 8 demo_movimiento temporizados y demo_fin.*

Con el WSS abierto se envió `POST /vehiculos/1/demos/1/iniciar` con la `X-API-Key`. La API respondió **202** (`estado: iniciada`, `total_movimientos`: 8) y ejecutó la DEMO en segundo plano. Los pasos se obtuvieron de `sp_obtener_movimientos_demo` y llegaron en orden por WSS: `demo_inicio`, ocho `demo_movimiento` y `demo_fin`. Cada movimiento incluye `orden`, `comando`, `codigo`, `duracion_ms` y `pausa_ms`, y el tiempo entre eventos respeta esa programación: por ejemplo, AVANZAR (2000 + 400 ms) llegó a las 09:38:07.136 y el siguiente paso a las 09:38:09.538, 2.4 s después.

Orden	Comando	duracion_ms	pausa_ms	Hora de recepción
1	AVANZAR	2000	400	09:38:07.136
2	GIRO_IZQUIERDA	1000	300	09:38:09.538
3	PALA_ARRIBA	1500	300	09:38:10.840
4	ROTACION_IZQUIERDA	900	250	09:38:12.641
5	ROTACION_DERECHA	900	250	09:38:13.793
6	PALA_ABAJO	1500	300	09:38:14.944
7	RETROCEDER	2000	400	09:38:16.746
8	DETENER	500	0	09:38:19.149

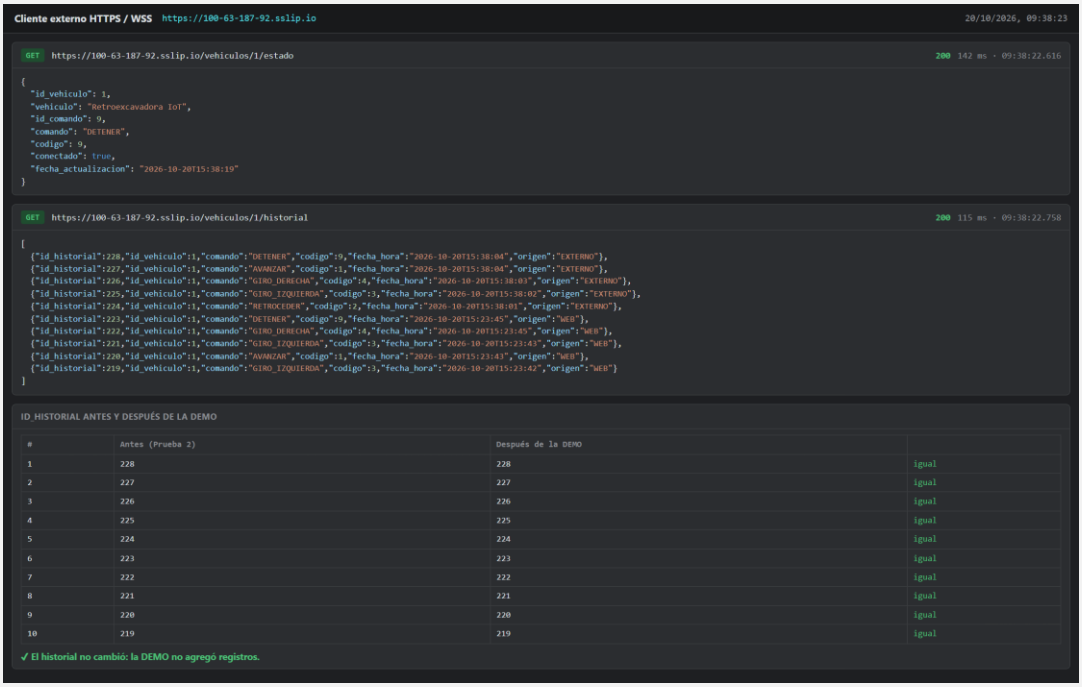


Figura 13. *Después de la DEMO: el estado quedó en DETENER y los id_historial son idénticos a los de la Prueba 2.*

Al terminar, `GET /vehiculos/1/estado` devolvió DETENER con `fecha_actualizacion` 15:38:19 UTC, la hora del último paso: durante la DEMO la API actualizó el estado con `sp_actualizar_estado`. El historial se consultó de nuevo y se comparó con el de la Prueba 2: los 10 `id_historial` (228 a 219) son exactamente los mismos, así que la DEMO **no agregó registros** a `historial_comandos`, porque nunca llama a `sp_registrar_historial`.

Simultaneidad, cancelación y seguridad

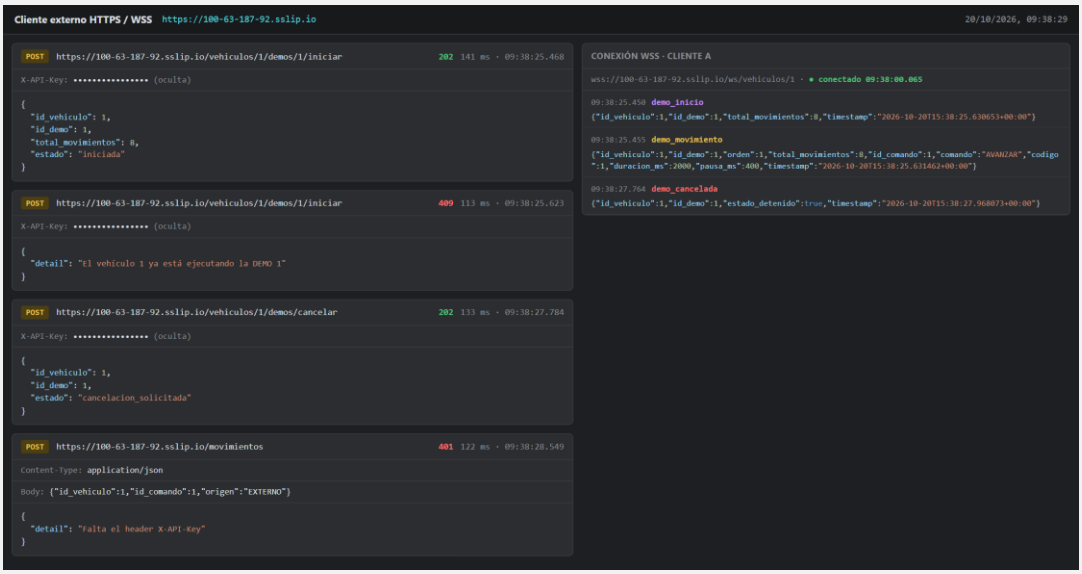


Figura 14. *Segunda DEMO simultánea rechazada (409), cancelación con evento demo_cancelada y POST sin X-API-Key rechazado (401).*

Se inició otra vez la DEMO 1 y, mientras corría, se intentó iniciarla de nuevo: la API respondió **409** ("El vehículo 1 ya está ejecutando la DEMO 1"), así que no hay dos DEMO simultáneas para el mismo vehículo. Después, `POST /vehiculos/1/demos/cancelar` respondió **202** y por WSS llegó `demo_cancelada` con `estado_detenido: true`, sin que se enviaran los pasos restantes. Por último, un `POST /movimientos` sin el header respondió **401** ("Falta el header X-API-Key") y no modificó nada. Estos eventos representan las futuras instrucciones para un ESP32/ESP8266, que solo tendrá que conectarse al mismo WSS y ejecutar cada `movimiento` o `demo_movimiento` durante `duracion_ms`.

4. Comprobación integral del despliegue

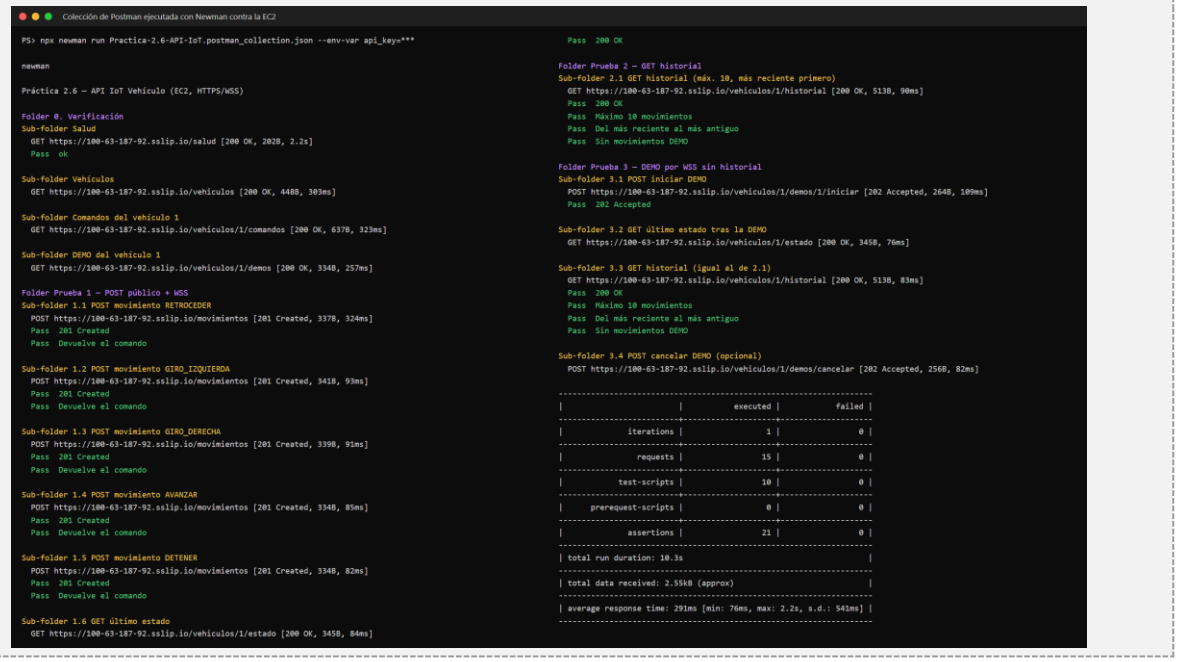


Figura 15. *Colección de Postman de la práctica ejecutada con Newman contra la EC2: 15 peticiones, 21 aserciones, 0 fallos.*

La colección `Practica-2.6-API-IoT.postman_collection.json` reúne la verificación inicial y las peticiones HTTP de las tres pruebas, con la URL pública como variable `base_url` y la llave como variable `api_key`. Se ejecutó completa con Newman contra la EC2: 15 peticiones y 21 aserciones sin fallos (201 en cada POST, máximo 10 movimientos, orden descendente y ningún movimiento DEMO en el historial). La parte WebSocket de la Prueba 3 se observa en el cliente WSS (Figura 12).

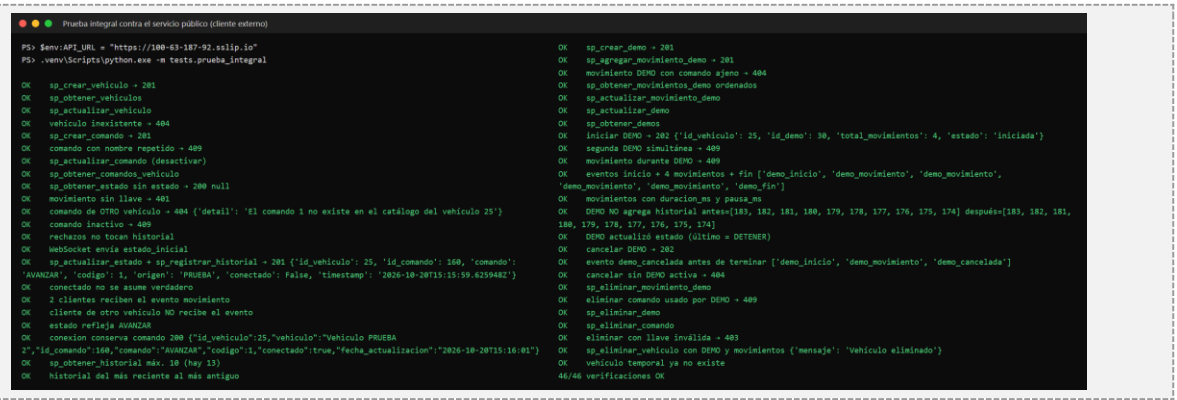


Figura 16. *Prueba integral del repositorio (tests/prueba_integral.py) contra https://100-63-187-92.sslip.io: 46/46 verificaciones correctas.*

Para comprobar que los 21 SP siguen disponibles funcionalmente a través de la API pública, se ejecutó la prueba integral del repositorio con `API_URL=https://100-63-187-92.sslip.io`. La prueba crea un vehículo temporal, recorre los procedimientos de vehículos, comandos, estado, historial, DEMO y movimientos DEMO, valida los errores (401 sin llave, 403 con llave inválida, 404 con un

comando de otro vehículo, 409 con un comando inactivo, una DEMO simultánea o un movimiento durante una DEMO), verifica el WSS con dos clientes y un tercero de otro vehículo que **no** recibe el evento, y al final elimina el vehículo temporal. El resultado fue **46/46 verificaciones OK**.

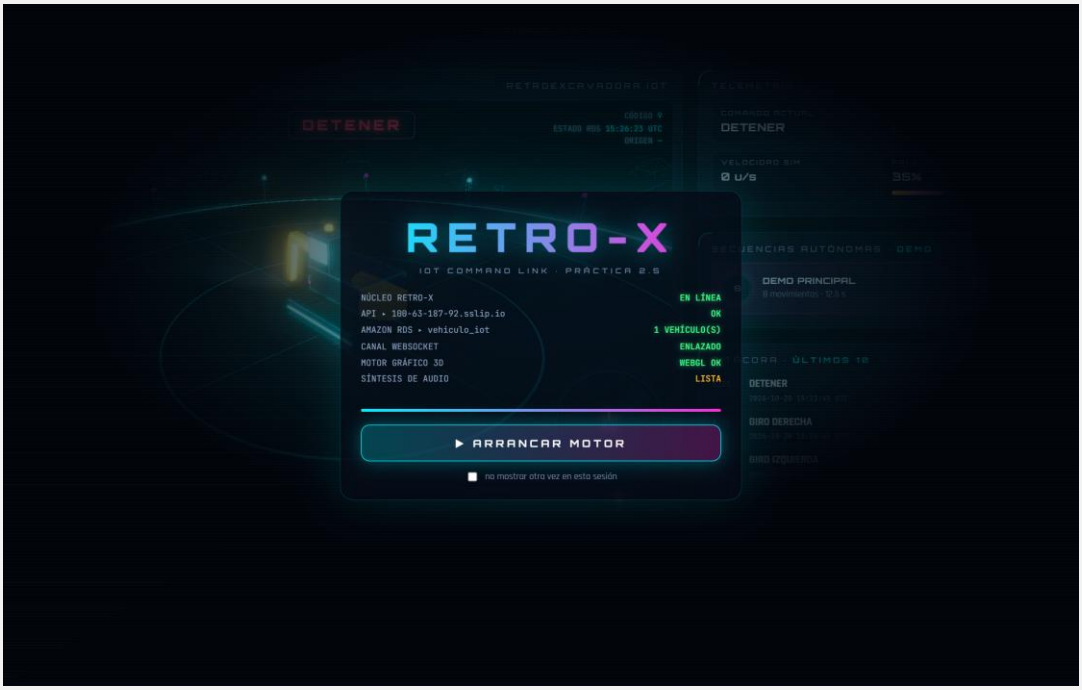


Figura 17. *Página de control pública https://100-63-187-92.sslip.io/control/, que usa la misma API y el mismo WSS.*

Además, la página de control servida por la misma API permite mover el vehículo con botones o teclado, ver el estado y el historial en vivo, lanzar o cancelar la DEMO y revisar el registro de eventos WebSocket. Funciona desde cualquier navegador sin instalar nada.

Evidencias mínimas

Evidencia	Contenido	Figuras
1. Arquitectura AWS y migración	Arquitectura EC2–RDS, ejecución persistente, conectividad con RDS, 21 SP y organización POO; datos sensibles ocultos	Figuras 1, 2 y 3
2. Swagger UI y Git/GitHub	URL pública HTTPS de Swagger UI, repositorio con el código y la documentación WSS	Figuras 4 a 7
3. POST + estado WSS	Estado inicial en dos conexiones WSS, POST individual, evento en ambos clientes y persistencia de estado e historial	Figuras 8, 9 y 10
4. GET historial	JSON público con 10 movimientos, del más reciente al más antiguo, sin DEMO	Figura 11
5. WSS y DEMO	Inicio, 8 movimientos con tiempos y fin; último estado y ausencia de registros DEMO en el historial; simultaneidad y cancelación	Figuras 12, 13 y 14

Evidencia	Contenido	Figuras
Comprobación	Colección de Postman (Newman), prueba integral 46/46 y página de control	Figuras 15, 16 y 17

Enlaces públicos para la evaluación

Swagger UI (HTTPS)	https://100-63-187-92.sslip.io/docs
Servicio WSS	`wss://100-63-187-92.sslip.io/ws/vehiculos/1`
API (HTTPS)	https://100-63-187-92.sslip.io
Página de control	https://100-63-187-92.sslip.io/control/
Repositorio GitHub	https://github.com/juanel2804/vehiculo-iot-api

Conclusiones

Programación orientada a objetos. Como la API ya estaba separada en configuración, acceso a datos, repositorios, servicios, ejecutor de DEMO, administrador WebSocket y routers, la migración no requirió cambiar el código de negocio: solo se movió la configuración a variables de entorno del servidor y se agregó la capa de despliegue (Caddy y tareas de inicio). Cada clase siguió teniendo una sola responsabilidad.

Los 21 Stored Procedures y Amazon RDS. Toda la lógica de datos vive en los procedimientos y la API los ejecuta con parámetros enlazados mediante un usuario que solo tiene permiso EXECUTE. Así, aunque la API esté expuesta a Internet, nadie puede leer ni modificar las tablas directamente a través de ella; solo se puede hacer lo que permiten los 21 SP.

GET/POST y publicación. Pasar de `localhost` a un servicio público implicó usar HTTPS y WSS con un certificado válido, dejar uvicorn escuchando solo en `127.0.0.1`, proteger las operaciones de control y eliminación con `X-API-Key` y lograr que el servicio arranque solo con la instancia. Las consultas GET nunca modifican datos y los POST escriben en RDS dentro de transacciones.

WebSocket. El WSS convierte la API en un canal de tiempo real: cada cliente recibe al conectarse el último estado y después solo los eventos de su vehículo, sin volver a consultar. En las pruebas, dos clientes recibieron cada movimiento con milisegundos de diferencia, lo que lo vuelve el canal natural para enviar instrucciones a un ESP32/ESP8266.

Movimiento individual frente a DEMO. El movimiento individual es una orden explícita de un usuario: actualiza el estado y queda en el historial como bitácora. La DEMO es una secuencia predefinida y repetible guardada en RDS: actualiza el estado en cada paso y se transmite con sus tiempos, pero no llena el historial con pasos automáticos. Por eso el historial sigue mostrando solo las acciones reales y la DEMO se controla aparte (una a la vez y cancelable).

Anexo A. Los 21 Stored Procedures y su endpoint

Stored Procedure	Endpoint de la API
sp_crear_vehiculo	POST /vehiculos
sp_obtener_vehiculos	GET /vehiculos
sp_obtener_vehiculo	GET /vehiculos/{id} (y validación en casi todo)
sp_actualizar_vehiculo	POST /vehiculos/{id}/actualizar
sp_eliminar_vehiculo	POST /vehiculos/{id}/eliminar
sp_crear_comando	POST /vehiculos/{id}/comandos
sp_obtener_comandos_vehiculo	GET /vehiculos/{id}/comandos
sp_actualizar_comando	POST /vehiculos/{id}/comandos/{id_comando}/actualizar
sp_eliminar_comando	POST /vehiculos/{id}/comandos/{id_comando}/eliminar
sp_actualizar_estado	POST /movimientos, POST /vehiculos/{id}/conexion y ejecución de DEMO
sp_obtener_estado	GET /vehiculos/{id}/estado
sp_registrar_historial	POST /movimientos
sp_obtener_historial	GET /vehiculos/{id}/historial
sp_crear_demo	POST /vehiculos/{id}/demos
sp_obtener_demos	GET /vehiculos/{id}/demos
sp_actualizar_demo	POST /vehiculos/{id}/demos/{id_demo}/actualizar
sp_eliminar_demo	POST /vehiculos/{id}/demos/{id_demo}/eliminar
sp_agregar_movimiento_demo	POST /vehiculos/{id}/demos/{id_demo}/movimientos
sp_obtener_movimientos_demo	GET .../demos/{id_demo}/movimientos y POST .../iniciar
sp_actualizar_movimiento_demo	POST .../movimientos/{id_mov}/actualizar
sp_eliminar_movimiento_demo	POST .../movimientos/{id_mov}/eliminar

Anexo B. Resumen visual de las 21 APIs

Hoja de resumen que reúne en una sola página la arquitectura (ESP32 → Internet HTTPS/WSS → Amazon EC2 con FastAPI → Amazon RDS), el flujo interno de cada API (router → servicio → repositorio → SP → RDS), los 21 Stored Procedures agrupados por área con su endpoint, cuerpo y respuestas, las operaciones que requieren `X-API-Key`, los códigos de respuesta, los eventos del WebSocket y la diferencia entre un movimiento individual y una DEMO. Nota: la cancelación de una DEMO (`POST /vehiculos/{id}/demos/cancelar`) responde **202 Accepted**, como se ve en la Figura 14.

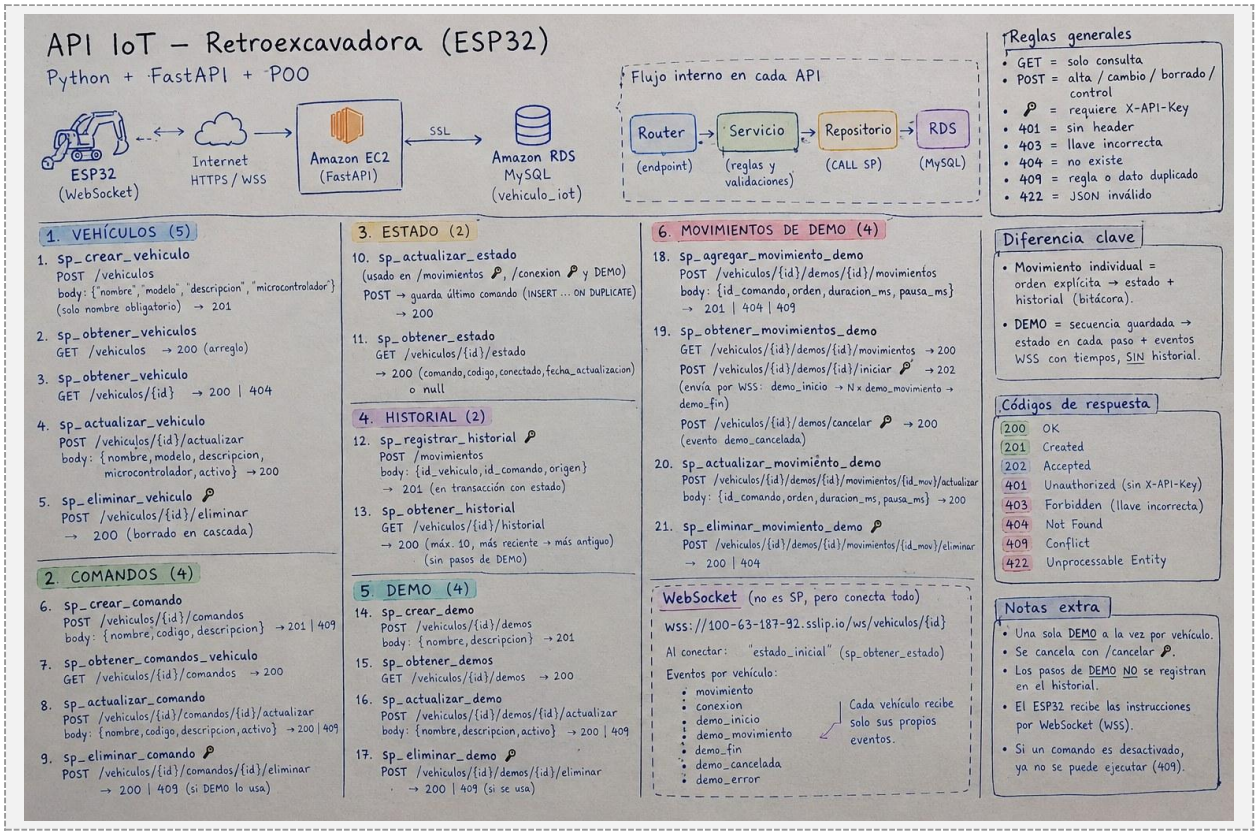


Figura 18. *Resumen visual de las 21 APIs: Stored Procedure, endpoint, cuerpo, respuestas y eventos WebSocket.*